

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESTRUCTURAS DE DATOS

CATEDRÁTICO: ING. ALVARO OBRAYAN HERNANDEZ GARCIA

TUTOR ACADÉMICO: LUIS ENRIQUE GARCIA GUTIERREZ



MANUAL TÉCNICO

Josué Daniel Fuentes Díaz

CARNÉ: 202300668

SECCIÓN: B

GUATEMALA, 03 DE OCTUBRE DEL 2,025

ÍNDICE

Contenido

ÍNDICE	1
INTRODUCCIÓN	2
OBJETIVOS.....	3
1. GENERAL	3
2. ESPECÍFICOS	3
ESPECIFICACIÓN TÉCNICA.....	4
• REQUISITOS DE HARDWARE	4
• REQUISITOS DE SOFTWARE	4
Unidades de Interfaz de Usuario (Forms)	5
➤ TfrmInbox.....	5
➤ TfrmUserN	5
➤ TfrmTrash.....	6
➤ TFormLogin	6
➤ TFormRoot	7
➤ TFormInbox.....	7
➤ TFormContacts	8
➤ TFormSendMail	8
➤ TFormTrash.....	9
➤ TFormProgramarMail.....	9
➤ TFormComunidades	10
➤ TfrmProgramarMail.....	10
➤ TfrmAddContact.....	11
➤ TfrmPerfil.....	11
Unidades de Estructuras de Datos	13
➤ uUsers	13
➤ uInbox.....	14
➤ uContacts.....	15
➤ uTrash.....	16
➤ uQueue	17
➤ uMatrix	18
➤ uComunidades.....	Error! Bookmark not defined.
➤ uComunidades.....	Error! Bookmark not defined.

INTRODUCCIÓN

El presente manual técnico documenta el desarrollo del sistema EDDMail, un proyecto académico de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala. El objetivo principal es simular un entorno de correo electrónico empleando estructuras de datos dinámicas implementadas en Object Pascal bajo el entorno Lazarus en Linux.

Este manual describe los objetivos del sistema, las especificaciones técnicas, los requisitos de hardware y software, así como la arquitectura lógica y las estructuras de datos utilizadas para la implementación.

OBJETIVOS

1. GENERAL

Desarrollar un sistema de simulación de correos electrónicos utilizando estructuras de datos dinámicas implementadas en Object Pascal que permita la gestión de usuarios, envío, recepción y administración de correos electrónicos.

2. ESPECÍFICOS

- Implementar una lista simple para almacenar y administrar los usuarios registrados en el sistema.
- Utilizar una lista doblemente enlazada para gestionar los correos recibidos en cada bandeja de entrada.
- Implementar una matriz dispersa para modelar y representar visualmente las relaciones entre emisores y receptores.

ESPECIFICACIÓN TÉCNICA

- **REQUISITOS DE HARDWARE**

- Procesador Mínimo 1 GHz o superior
- Memoria RAM mínima: 4 GB
- Al menos 500 MB de espacio disponible para la instalación y ejecución del programa

- **REQUISITOS DE SOFTWARE**

- Sistema Operativo: Linux (Debian 12 o superior, Ubuntu 22.04 o superior).
- IDE: Lazarus 3.0 o superior.
- Compilador: Free Pascal (FPC 3.2.2 o superior)
- Bibliotecas:
 - GTK2/GTK3 para la interfaz gráfica
 - Graphviz para la generación de reportes gráficos (usuarios, relaciones y comunidades).

Unidades de Interfaz de Usuario (Forms)

➤ TfrmInbox

- Descripción: Clase principal para la gestión de la bandeja de entrada de correos.
- Funcionalidad: Maneja la visualización, ordenamiento y eliminación de correos en la bandeja de entrada del usuario

```
{ TfrmInbox }

TfrmInbox = class(TForm)
    lstMails: TListBox;           // lista (Estado | Asunto | Remitente)
    lblUnread: TLabel;           // "No leídos: N"
    btnOrdenarAZ: TButton;        // ordena por Asunto A-Z
    btnEliminar: TButton;         // elimina de la bandeja
    btnRegresar: TButton;         // vuelve al menú de usuario

    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure lblUnreadClick(Sender: TObject);
    procedure lstMailsClick(Sender: TObject);
    procedure btnOrdenarAZClick(Sender: TObject);
    procedure btnEliminarClick(Sender: TObject);
    procedure btnRegresarClick(Sender: TObject);
    procedure lstMailsDbClick(Sender: TObject);

private
    procedure RefrescarLista;
    function IndexValido: Boolean;
end;
```

➤ TfrmUserN

- Descripción: Clase del dashboard principal del usuario autenticado.
- Funcionalidad: Proporciona navegación centralizada a todas las funciones del sistema de correo.

```
type
{ TfrmUserN }

TfrmUserN = class(TForm)
    btnBandeja: TButton;
    btnCargaMasiva1: TButton;
    btnPapeleria: TButton;
    btnProgramarCorreo: TButton;
    btnVerProgramados: TButton;
    btnAgregarContacto: TButton;
    btnContactos: TButton;
    btnPerfil: TButton;
    btnReportes: TButton;
    btnCerrarSesion: TButton;
    lblWelcome: TLabel;
    procedure btnAgregarContactoClick(Sender: TObject);
    procedure btnBandejaClick(Sender: TObject);
    procedure btnCargaMasiva1Click(Sender: TObject);
    procedure btnReportesClick(Sender: TObject);
    procedure btnContactosClick(Sender: TObject);
    procedure btnPapeleriaClick(Sender: TObject);
    procedure btnCerrarSesionClick(Sender: TObject);
    procedure btnPerfilClick(Sender: TObject);
    procedure btnProcesarProgramadosClick(Sender: TObject);
    procedure btnProgramarCorreoClick(Sender: TObject);
    procedure btnVerProgramadosClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);

private
public
end;
```

➤ TfrmTrash

- Descripción: Clase para la gestión de la papelera de correos eliminados.
- Funcionalidad: Permite visualizar, buscar y eliminar permanentemente correos de la papelera.

```
{ TfrmTrash }

TfrmTrash = class(TForm)
    lblTitulo: TLabel;
    edtBuscar: TEdit;
    btnBuscar: TButton;
    btnEliminar: TButton;
    btnRegresar: TButton;
    lstTrash: TListBox;
    procedure FormShow(Sender: TObject);
    procedure btnBuscarClick(Sender: TObject);
    procedure btnEliminarClick(Sender: TObject);
    procedure btnRegresarClick(Sender: TObject);
private
    FUltimoFiltro: string;
    procedure LlenarLista(const FiltroAsunto: string = '');
    procedure EliminarPorIndiceDesdeTop(IndexFromTop: Integer);
end;
```

➤ TFormLogin

- Descripción: Formulario principal de inicio de sesión.
- Funcionalidad: Permite ingresar credenciales de usuario (email y contraseña). Valida si el usuario existe y redirige al menú correspondiente (Root o estándar)

```
type
{ TFormLogin }

TfrmLogin = class(TForm)
    CrearCuenta: TButton;
    Label1: TLabel;
    lblEmail: TLabel;
    edtEmail: TEdit;
    lblPass: TLabel;
    edtPass: TEdit;
    btnIngresar: TButton;

    procedure CrearCuentaClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);
    procedure btnIngresarClick(Sender: TObject);
private
    procedure AbrirMain;
public
end;
```

➤ TFormRoot

- Descripción: Formulario exclusivo para el usuario administrador (root).
- Funcionalidad: Ofrece opciones como carga masiva de usuarios desde archivo JSON, generación de reportes de usuarios y reportes de relaciones (matriz dispersa)

```
{ TfrmRoot }

TfrmRoot = class(TForm)
published
    btnExportRel: TButton;
    btnExportUsers: TButton;
    btnComunidades: TButton;
    btnExportComunidades: TButton;
    btnSalir: TButton;
    btnCargaMasiva: TButton;
    Label1: TLabel;
    procedure btnCargaMasivaClick(Sender: TObject);
    procedure btnComunidadesClick(Sender: TObject);
    procedure btnExportComunidadesClick(Sender: TObject);
    procedure btnExportRelClick(Sender: TObject);
    procedure btnExportUsersClick(Sender: TObject);
    procedure btnSalirClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);
public
    procedure Button1Click(Sender: TObject);
    procedure ActionCrearComunidad;
    procedure ActionAgregarUsuarioAComunidad;
    procedure ActionExportarComunidades;
end;
```

➤ TFormInbox

- Descripción: Formulario de la bandeja de entrada.
- Funcionalidad: Muestra los correos recibidos (lista doblemente enlazada). Permite ordenarlos por asunto, marcarlos como leídos/no leídos y eliminarlos (pasan a la pila de la papelera)

```
{ TfrmInbox }

TfrmInbox = class(TForm)
    lstMails: TListBox; // lista (Estado | Asunto | Remitente)
    lblUnread: TLabel; // "No leídos: N"
    btnOrdenarAZ: TButton; // ordena por Asunto A-Z
    btnEliminar: TButton; // elimina de la bandeja
    btnRegresar: TButton; // vuelve al menú de usuario
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure lblUnreadClick(Sender: TObject);
    procedure lstMailsClick(Sender: TObject);
    procedure btnOrdenarAZClick(Sender: TObject);
    procedure btnEliminarClick(Sender: TObject);
    procedure btnRegresarClick(Sender: TObject);
    procedure lstMailsDbClick(Sender: TObject);
private
    procedure RefrescarLista;
    function IndexValido: Boolean;
end;
```

➤ TFormContacts

- Descripción: Formulario de contactos de un usuario.
- Funcionalidad: Maneja la lista circular de contactos. Permite agregar, visualizar y navegar entre contactos registrados en el sistema

```
type
[
  { TfrmContacts }

  TfrmContacts = class(TForm)
    btnPrev: TButton;
    btnRergresar: TButton;
    lblAsunto: TLabel;
    lblAsunto1: TLabel;
    lblAsunto2: TLabel;
    lblPara: TLabel;
    lblTitulo: TLabel;
    btnPrev: TButton;
    btnNext: TButton;
    // labels de valores:
    valNombre: TLabel;
    valUsuario: TLabel;
    valCorreo: TLabel;
    valTelefono: TLabel;
    procedure btnRergresarClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnPrevClick(Sender: TObject);
    procedure btnNextClick(Sender: TObject);
  private
    procedure MostrarActual;
    procedure HabilitarNavegacion(const OnOff: Boolean);
  public
  end;
end;
```

➤ TFormSendMail

- Descripción: Ventana de redacción de correos.
- Funcionalidad: Permite redactar y enviar un correo a un contacto válido. También sirve para programar correos con fecha/hora (se almacenan en la cola de correos programados)

```
[
  { TfrmSendMail }

  TfrmSendMail = class(TForm)
    lblParaa: TLabel; lblAsunto: TLabel; lblMensaje: TLabel;
    edtPara: TEdit; edtAsunto: TEdit;
    memMensaje: TMemo;
    btnEnviar: TButton; btnCancelar: TButton;
    procedure FormCreate(Sender: TObject);
    procedure btnEnviarClick(Sender: TObject);
    procedure btnCancelarClick(Sender: TObject);
  end;
end;
```

➤ TFormTrash

- Descripción: Ventana de correos eliminados.
- Funcionalidad: Muestra la pila de correos eliminados. Permite buscar por palabra clave en el asunto o eliminarlos definitivamente

```
TfrmTrash = class(TForm)
    lblTitulo: TLabel;
    edtBuscar: TEdit;
    btnBuscar: TButton;
    btnEliminar: TButton;
    btnRegresar: TButton;
    lstTrash: TListBox;
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnBuscarClick(Sender: TObject);
    procedure btnEliminarClick(Sender: TObject);
    procedure btnRegresarClick(Sender: TObject);
private
    FUltimoFiltro: string;
    procedure LlenarLista(const FiltroAsunto: string = '');
    procedure EliminarPorIndiceDesdeTop(IndexFromTop: Integer);
end;
```

➤ TFormProgramarMail

- Descripción: Ventana de gestión de correos programados.
- Funcionalidad: Muestra los correos pendientes en la cola. Permite enviarlos manualmente o esperar que se envíen automáticamente

```
{ TfrmProgramarMail }

TfrmProgramarMail = class(TForm)
    lblPara: TLabel;
    lblAsunto: TLabel;
    lblFecha: TLabel;
    lblHora: TLabel;
    edtPara: TEdit;
    edtAsunto: TEdit;
    memMensaje: TMemo;
    edtFecha: TEdit; // yyyy-mm-dd
    edtHora: TEdit; // hh:nn
    btnProgramar: TButton;
    btnCancelar: TButton;
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnProgramarClick(Sender: TObject);
    procedure btnCancelarClick(Sender: TObject);
private
    function ParseDateTimeSafe(const SDate, STime: string; out DT: TDateTime): Boolean;
public
end;
```

➤ TFormComunidades

- Descripción: Ventana de gestión de comunidades (lista de listas).
- Funcionalidad: Exclusiva del usuario root. Permite crear comunidades y asignar usuarios a cada una

```
{ TfrmComunidades }
TfrmComunidades = class(TForm)
    btnCrear: TButton;
    btnAgregar: TButton;
    btnSalir: TButton;
    edNombreComunidad: TEdit;
    edComunidad: TEdit;
    edCorreo: TEdit;
    lblNombre1: TLabel;
    lblTitulo: TLabel;
    lblNombre: TLabel;
    lblComunidad: TLabel;
    lblCorreo: TLabel;
    procedure btnCrearClick(Sender: TObject);
    procedure btnAgregarClick(Sender: TObject);
    procedure btnSalirClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
private
    function BuscarNombrePorCorreo(const Email: String): String;
    procedure Msg(const s: String);
public
end;
```

➤ TfrmProgramarMail

- Descripción: Programación de correos.
- Funcionalidad: Permite programar envío de correos indicando fecha y hora; se almacenan en una cola

```
type
{ TfrmProgramarMail }
TfrmProgramarMail = class(TForm)
    lblPara: TLabel;
    lblAsunto: TLabel;
    lblFecha: TLabel;
    lblHora: TLabel;
    edtPara: TEdit;
    edtAsunto: TEdit;
    memMensaje: TMemo;
    edtFecha: TEdit; // yyyy-mm-dd
    edtHora: TEdit; // hh:nn
    btnProgramar: TButton;
    btnCancelar: TButton;
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnProgramarClick(Sender: TObject);
    procedure btnCancelarClick(Sender: TObject);
private
    function ParseDateTimeSafe(const SDate, STime: string; out DT: TDateTime): Boolean;
public
end;
```

➤ TfrmAddContact

- Descripción: Formulario de agregar contacto.
- Funcionalidad: Permite añadir un nuevo contacto mediante su email

type

```
{ TfrmAddContact }

TfrmAddContact = class(TForm)
    lblTitulo: TLabel;
    edtEmail: TEdit;
    btnAgregar: TButton;
    btnCancelar: TButton;
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnAgregarClick(Sender: TObject);
    procedure btnCancelarClick(Sender: TObject);
end;
```

➤ TfrmPerfil

- Descripción: Perfil de usuario.
- Funcionalidad: Permite actualizar datos del usuario como nombre o teléfono

type

```
{ TfrmPerfil }

TfrmPerfil = class(TForm)
    edtUsuarioE: TLabel;
    lblPara1: TLabel;
    lblTitulo: TLabel;
    lblUsuario: TLabel;
    lblTelefono: TLabel;
    edtUsuario: TEdit;
    edtTelefono: TEdit;
    btnActualizar: TButton;
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnActualizarClick(Sender: TObject);
private
    procedure CargarDatosActuales;
public
end;
```

➤ TFormBorradores

- Descripción: Correos mandados a borradores.
- Funcionalidad: Permite guardar los correos en borradores para ser enviados más tarde

```

1  unit FBorradores;
.
.  {$mode ObjFPC}{$SH+}
.
.
5  interface
.
.  uses
.      Classes, SysUtils, Forms, Controls, Dialogs, Grids, StdCtrls,
.      UAVL_Borradores, UDataAVL, frmUser;
10
.  type
.      { TFormBorradores }
.      TFormBorradores = class(TForm)
.          btnRefrescar: TButton;
15         btnAbrir: TButton;
.          btnEliminar: TButton;
.          btnRegresar: TButton;
.          sgBorradores: TStringGrid;
.          procedure btnRegresarClick(Sender: TObject);
20         procedure FormCreate(Sender: TObject);
.          procedure btnRefrescarClick(Sender: TObject);
.          procedure btnAbrirClick(Sender: TObject);
.          procedure btnEliminarClick(Sender: TObject);
.          procedure FormShow(Sender: TObject);
25
.      private
.          procedure LlenarGrid;
.          function IDSeleccionado(out AID: LongInt): boolean;
.          procedure VisitAddToGrid(const D: TMailDraft);
30         var Ffila: Integer;
.      public
.          end;
.

```

➤ TfrmFavoritos

- Descripción: Perfil de usuario.
- Funcionalidad: Permite actualizar datos del usuario como nombre o teléfono

```

implementation
.
.  {$R *.lfm}
.
.  uses fSendMail;
.
.  procedure TFormBorradores.FormCreate(Sender: TObject);
.  begin
.      Caption := 'Borradores';
.      sgBorradores.ColCount := 4;
.      sgBorradores.RowCount := 1;
.      sgBorradores.Cells[0,0] := 'ID';
.      sgBorradores.Cells[1,0] := 'Remitente';
.      sgBorradores.Cells[2,0] := 'Destinatario';
.      sgBorradores.Cells[3,0] := 'Asunto';
.  end;
.
.  procedure TFormBorradores.btnRegresarClick(Sender: TObject);
.  begin
.      FormBorradores.Hide;
.      frmUserN.Show;
.  end;
.
.  procedure TFormBorradores.LlenarGrid;
.  begin
.      Ffila := 1;
.      sgBorradores.RowCount := 1;
.      AVL_InOrder(BorradoresRoot, @VisitAddToGrid); // método del form
.  end;
.
.  procedure TFormBorradores.FormShow(Sender: TObject);
.  begin
.      LlenarGrid;           // refresca cada vez que se muestra
.  end;
.
.  procedure TFormBorradores.VisitAddToGrid(const D: TMailDraft);
.

```

Unidades de Estructuras de Datos

➤ uUsers

- Descripción: Gestión central de usuarios mediante lista enlazada simple.
- Estructuras:
 - TUserNode: Registro principal de usuario con ID autoincremental, datos personales y estructuras asociadas (Inbox, Trash, Contacts, Sched)
 - Lista enlazada simple con inserción O(1) al inicio
- Funcionamiento:
 - Autoincremento de IDs únicos
 - Búsqueda por email o username
 - Validación de login con determinación de tipo de usuario
 - Carga masiva desde JSON con manejo de duplicados
 - Exportación a formato DOT para visualización

```
unit uUsers;

{$mode objfpc}{$SH+}

interface
uses uInbox, uTrash, uQueue, uContacts;

function LoadUsersFromJSON(const FileName: string;
                           out Imported, Duplicated, Errors: Integer): Boolean;
function ExistsId(const AId: Integer): Boolean;
function AddUserWithId(const AId: Integer; const AName, AUsername, AEmail, APhone, APass: AnsiString;
                      const ARoot: Boolean): Integer;
function ExportUsersDot(const DirPath: string): Boolean;
function ExportContactsDOTForUser(const L: TContactList;
                                  const OwnerEmail, BaseDir: string;
                                  out DotPath: string): Boolean;

|

type
  PUser = ^TUserNode;
  TUserNode = record
    Id: Integer;           // autoincremental
    Name: AnsiString;     // Nombre
    Username: AnsiString; // Usuario
    Email: AnsiString;    // Email
    Phone: AnsiString;    // Teléfono
    Password: AnsiString; // Contraseña
    IsRoot: Boolean;      // Root?
    Inbox: TInbox;        // Bandeja del user
    Next: PUser;          // Siguiente
    Trash: TTrash;
    Contacts: TContactList;
    Sched: TSchedQueue;   //Cola de correos programados
  end;
```

➤ uInbox

- Descripción: Bandeja de entrada como lista doblemente enlazada.
- Estructuras:
 - TMail: Correo con enlaces bidireccionales (Prev/Next)
 - TInbox: Contenedor con punteros Head/Tail y contador
- Funcionamiento:
 - Inserción al final O(1)
 - Navegación bidireccional
 - Ordenamiento por asunto mediante insertion sort
 - Marcado de lectura y extracción de elementos
 - Exportación estructural a DOT

```
unit uInbox;

{$mode objfpc}{$SH+}

interface

type
  PMail = ^TMail;
  TMail = record
  // enlaces
  Prev, Next : PMail;
  // datos del correo
  Id          : Integer;
  Remitente   : AnsiString;
  Estado      : ShortString; // 'NL' / 'L'
  Programado  : Boolean;
  Asunto      : AnsiString;
  Fecha       : AnsiString;
  Mensaje     : AnsiString;
end;

  TInbox = record
  Head, Tail : PMail;
  Count      : Integer;
end;

var
  NextMailId: Integer = 1;

procedure InitInbox(var B: TInbox);
function AddMail(var B: TInbox; const ARem, AAsunto, AFecha, AMensaje: AnsiString;
               const AProg: Boolean): PMail;
function CountUnread(const B: TInbox): Integer;
function GetMailByIndex(const B: TInbox; Index: Integer): PMail;
procedure MarkRead(M: PMail);
procedure DetachMail(var B: TInbox; M: PMail);
procedure SortBySubject(var B: TInbox);
// NUEVO
function ExtractMailAt(var I: TInbox; Index: Integer): PMail;
function ExportInboxDOTForUser(const UserEmail: string; const B: TInbox;
                             const BaseDir: string; out DotPath: string): Boolean;
```

➤ uContacts

- Descripción: Lista circular simple de contactos.
- Estructuras:
 - TContact: Nodo con información de contacto y enlace Next
 - TContactList: Tail apunta al último, Curr para navegación UI
- Funcionamiento:
 - Inserción al final manteniendo circularidad
 - Navegación circular en ambas direcciones
 - Prevención de duplicados por email/username
 - Contador de elementos para control de navegación

```
unit uContacts;  
  
{$mode objfpc}{$SH+}  
  
interface  
  
type  
  PContact = ^TContact;  
  TContact = record  
    Name      : AnsiString;  
    Username  : AnsiString;  
    Email     : AnsiString;  
    Phone     : AnsiString;  
    Next      : PContact; // lista **circular** simple  
  end;  
  
  // Lista circular con puntero a cola (Tail^.Next es la cabeza)  
  TContactList = record  
    Tail : PContact; // nil si está vacía  
    Curr : PContact; // para navegar (UI)  
    Count: Integer;  
  end;  
  
procedure InitContacts(var L: TContactList);  
procedure ClearContacts(var L: TContactList);  
  
// Inserta al final si no existe (por email/username). Devuelve True si se agregó.  
function AddContact(var L: TContactList; const AName, AUser, AEmail, APhone: AnsiString): Boolean;  
  
// ¿Existe un contacto por email o username?  
function ExistsInContacts(const L: TContactList; const Key: AnsiString): Boolean;  
  
// Iteración para la UI (siguiente / anterior en la lista circular)  
function HeadContact(const L: TContactList): PContact;  
function NextContact(var L: TContactList): PContact; // avanza Curr y lo retorna  
function PrevContact(var L: TContactList): PContact; // retrocede Curr y lo retorna
```

➤ uTrash

- Descripción: Papelera como pila LIFO.
- Estructuras:
 - TTrashNode: Nodo contenedor de PMail
 - TTrash: Pila con puntero Top y contador
- Funcionamiento:
 - Operaciones Push/Pop estándar LIFO
 - Reutilización de estructura PMail existente
 - Peek no destructivo
 - Verificación de estado vacío

```
unit uTrash;  
  
{$mode objfpc}{$H+}  
  
interface  
  
uses  
    uInbox; // para el tipo PMail (puntero al correo)  
  
type  
    PTrashNode = ^TTrashNode;  
    TTrashNode = record  
        Mail: PMail; // puntero al correo eliminado  
        Next: PTrashNode; // siguiente en la pila  
    end;  
  
    TTrash = record  
        Top: PTrashNode; // tope de la pila  
        Count: Integer; // cantidad de elementos  
    end;  
  
procedure InitTrash(out T: TTrash);  
procedure PushTrash(var T: TTrash; M: PMail);  
function PopTrash(var T: TTrash): PMail;  
function PeekTrash(const T: TTrash): PMail;  
function IsTrashEmpty(const T: TTrash): Boolean;
```

➤ uQueue

- Descripción: Cola FIFO para correos programados.
- Estructuras:
 - TSchedItem: Elemento de cola con datos del correo
 - TSchedQueue: Cola con punteros Head/Tail
- Funcionamiento:
 - Encolado al final (Tail)
 - Procesamiento FIFO completo
 - Conversión a correos reales en bandeja de destino
 - Exportación de estructura a DOT

```
unit uQueue;

{$mode objfpc}{$SH+}

interface

type
  PSchedItem = ^TSchedItem;
  TSchedItem = record
    Dest      : AnsiString;    // email o username del destinatario
    Asunto    : AnsiString;
    Fecha     : AnsiString;    // solo para mostrar; ya no controla nada
    Mensaje   : AnsiString;
    Next      : PSchedItem;
  end;

  TSchedQueue = record
    Head, Tail : PSchedItem;
    Count      : Integer;
  end;

procedure InitQueue(var Q: TSchedQueue);
procedure ClearQueue(var Q: TSchedQueue);

// Encola al final (FIFO)
procedure EnqueueScheduled(var Q: TSchedQueue; const ADest, AAsunto, AFecha, AMsg: AnsiString);

// PROCESA TODO en orden FIFO. Devuelve cuántos envió.
function ProcessFIFO(var Q: TSchedQueue): Integer;

// Reporte DOT de la cola del usuario (no muta la cola)
function ExportSchedDOTForUser(const Q: TSchedQueue;
  const UserEmail, BaseDir: string;
  out DotPath: string): Boolean;
```

➤ uMatrix

- Descripción: Matriz dispersa ortogonal para relaciones de envío.
- Estructuras:
 - TRelNode: Nodo con enlaces ortogonales (Right/Left, Down/Up)
 - THead: Cabeceras ordenadas por ID de usuario
 - TRelMatrix: Contenedor con cabeceras de fila y columna
- Funcionamiento:
 - Inserción ordenada en filas y columnas
 - Incremento automático de contadores de relación
 - Búsqueda eficiente por coordenadas
 - Exportación compleja a DOT con alineación matricial

```
unit uMatrix;

{$mode objfpc}{$SH+}

interface

uses
  uUsers; // PUser

const
  OUT_DIR_REL = 'Root-Reportes/'; // carpeta para reportes de relaciones

type
  PRelNode = ^TRelNode;
  TRelNode = record
    RowUser : PUser; // remitente (fila)
    ColUser : PUser; // destinatario (columna)
    Count : Integer; // # correos enviados Row->Col
    // enlaces ortogonales
    Right, Left : PRelNode;
    Down, Up : PRelNode;
  end;

  PHead = ^THead;
  THead = record
    User : PUser; // identifica fila o columna
    Next : PHead; // cabeceras ordenadas por Id (o por puntero)
    Node : PRelNode; // primer nodo de esa fila/columna
  end;

  TRelMatrix = record
    RowHeads : PHead; // cabeceras por remitente
    ColHeads : PHead; // cabeceras por destinatario
    Total : Integer; // # celdas (pares con al menos 1 envío)
  end;

var
  RelMatrix: TRelMatrix; // matriz global de relaciones

procedure InitRelMatrix(var M: TRelMatrix);
procedure InitRel; // inicializa la global
```

- **UAVL_Borradores**
- Descripción: Gestión de correos guardados como borradores mediante un árbol AVL balanceado.
- Estructuras:
 - TMailDraft: Registro del borrador
 - TAVLNode: Nodo del árbol AVL
 - TVisitDraftProc: Procedimiento de callback para recorridos
- Funcionamiento:
 - Inserción de borradores con balanceo automático
 - Eliminación de borradores manteniendo las propiedades AVL
 - Recorridos disponibles para visualización
 - Generación de ID única para cada borrador

```

unit UAVL_Borradores;

{$mode ObjFPC}{$H+}

interface

type
  // Registro de borrador (evitamos chocar con tus tipos de Fase 1)
  PMailDraft = ^TMailDraft;
  TMailDraft = record
    ID      : LongInt;
    Remitente : AnsiString;
    Destinatario: AnsiString;
    Asunto    : AnsiString;
    Mensaje   : AnsiString;
  end;

  type
    TVisitDraftProc = procedure(const D: TMailDraft) of object;

  // Nodo del AVL
  PAVLNode = ^TAVLNode;
  TAVLNode = record
    KeyID : LongInt;      // Usamos el ID del correo como clave
    Data  : TMailDraft;   // El borrador completo
    Height: SmallInt;     // Altura del nodo
    Left, Right: PAVLNode; // Hijos
  end;

  // API mínima que vas a usar
  procedure AVL_Init(var Root: PAVLNode);
  function  AVL_Insert(var Root: PAVLNode; const D: TMailDraft): boolean; // false si duplicado (ID repetido)
  function  AVL_Find(Root: PAVLNode; ID: LongInt; out D: TMailDraft): boolean;
  function  AVL_Delete(var Root: PAVLNode; ID: LongInt): boolean;

  procedure AVL_InOrder(Root: PAVLNode; Visit: TVisitDraftProc);
  procedure AVL_PreOrder(Root: PAVLNode; Visit: TVisitDraftProc);
  procedure AVL_PostOrder(Root: PAVLNode; Visit: TVisitDraftProc);
  function  AVL_Count(Root: PAVLNode): LongInt;

```

- **UDataAVL**
- Descripción: Módulo que proporciona la gestión global de la raíz del AVL de borradores y utilidades auxiliares.
- Estructuras:
 - BorradoresRoot: Raíz global del árbol AVL donde se almacenan todos los borradores en memoria
 - DraftsFilePath: Ruta del fichero donde se guardan/recargan los borradores al cerrar/abrir la aplicación
 - _Seq: Contador interno para NextDraftID
- Funcionamiento:
 - En initialization se llama AVL_Init(BorradoresRoot) y se fija DraftsFilePath por defecto
 - En finalization se llama SaveDraftsToFile(DraftsFilePath) para persistir el estado actual del AVL antes de cerrar la aplicación
 - NextDraftID incrementa _Seq y devuelve el valor

```

unit UDataAVL;

{$mode ObjFPC}{$SH+}

interface
uses UAVL_Borradores;

var
  BorradoresRoot: PAVLNode; // raíz del AVL (global)

function NextDraftID: LongInt;

implementation

var
  _Seq: LongInt = 1;

function NextDraftID: LongInt;
begin
  Inc(_Seq);
  Result := _Seq;
end;

initialization
  AVL_Init(BorradoresRoot);
finalization
  // (opcional) liberar nodos si lo deseas
end.

```
